IMPLEMENTATION BASELINE

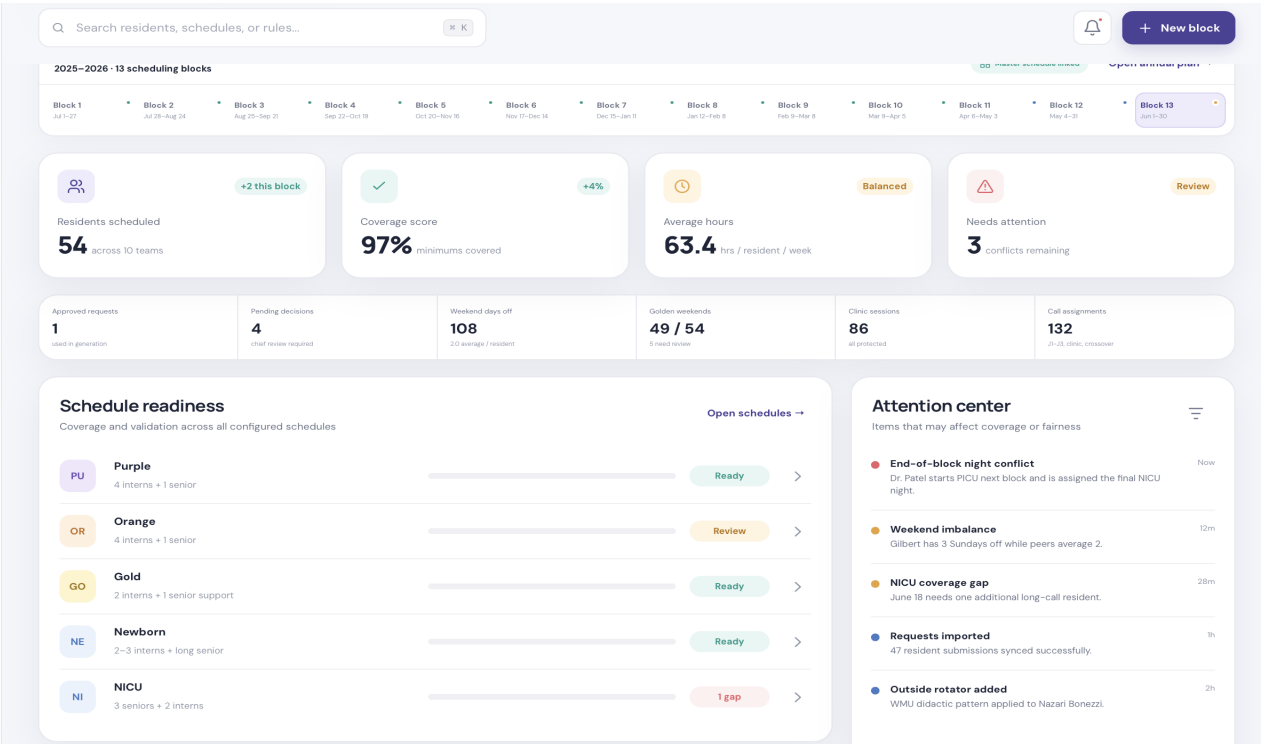
Clarity Schedule

Business Requirements Document and Software Requirements Specification

Document attribute	Value
Version	1.0 implementation baseline
Date	June 10, 2026
Prepared for	Program leadership, chief residents, implementation team, and institutional stakeholders
Status	Draft for validation and implementation planning

Purpose. Define the processes, behavior, data, rules, controls, and delivery plan required to move the concept from prototype to production.

Governance. The system recommends and validates; authorized program leadership approves policies, exceptions, and published schedules.



1. Executive Summary

Clarity Schedule is a configurable residency scheduling platform with separate chief and resident portals. It connects annual rotation planning, resident availability, institution patterns, service rules, monthly generation, editable schedules, approvals, publication, analytics, and call exchanges.

The product promise is to enter reliable facts once, collect requests in one place, generate explainable drafts, allow chief editing, and synchronize every downstream view.

2. Business Problems Solved

Fragmented forms, spreadsheets, PDFs, messages, and institutional calendars create duplicate work and missed constraints.

Each service has different staffing, role, unit, shift, night, weekend, and call requirements.

Manual fairness and call-switch review is slow and difficult to reproduce.

Annual rotation decisions and monthly service schedules are not reliably linked.

3. Users and Access

Chief administrators configure, generate, edit, approve, publish, and report.

Coordinators maintain profiles, imports, events, deadlines, and communications under delegated permissions.

Residents see their own private schedule and requests plus read-only published service rosters.

Program directors and read-only leaders receive oversight access without unnecessary private details.

4. End-to-End Workflow

Configure program, blocks, rotations, services, roles, shifts, institutions, and rules.

Import residents and collect annual and block requests.

Approve requests, build the annual master schedule, and validate capacity/transitions.

Pull each service roster from the selected master-schedule block and generate independent drafts.

Edit schedules with live coverage, workload, fairness, and protected-time checks.

Publish controlled versions and support resident call-switch proposals with chief approval.

5. Functional Capability Map

Domain	Required capability
Program configuration	Custom years, block dates, request deadlines, services, rotations, roles, shifts, locations, and colors.
Profiles	Resident facts, institution, PGY, clinic, didactics, eligibility, leave, and exceptions.
Requests	Weekend, day off, vacation, elective, fellowship, PTO, sick leave, special circumstances, and decision workflow.
Master schedule	Editable annual grid, capacities, requirements, preference scoring, locks, conflicts, and versions.
Service scheduling	Master-linked roster, per-service generator, four-week editor, call pool, live checks, undo/redo, and versions.
Resident portal	Private annual/monthly schedule, calls, protected time, requests, PTO, and published rosters.
Call switches	Offer marketplace, eligibility checks, volunteer, chief decision, and atomic correction.
Analytics	Hours, calls, nights, weekends, golden weekends, clinics, role coverage, fairness, and readiness.

6. Requirement Baseline

Area	Mandatory requirement
Identity and access	Institutional authentication, role-based permissions, private resident boundaries, tenant/program segregation, and full audit history.
Blocks	13, 26, monthly, or custom block models; deadlines; holidays; protected program events; lifecycle status.
Institutions	Reusable clinic/didactic patterns, buffers, eligibility, automatic inheritance, and individual/block overrides.
Services	Custom services mapped to master rotations with locations, roles, counts, shifts, hours, colors, and versioned rules.
Requests	Only approved requests constrain generation; competing requests receive advisory priority information and chief decision.
Master schedule	One rotation per resident/block, capacity and annual requirements, preference context, transition checks, locks, and versions.
Generation	Hard constraints first, soft optimization second, explicit infeasibility, explainable selection, and supplemental eligibility.
Editing	Drag, move, copy, clear, custom assignments, add/remove resident, undo/redo, autosave, versions, notes, and live recalculation.
Publishing	Ready/published/corrected states, immutable snapshots, selective publication, resident notification, PDF/XLSX export.
Call switches	Clinic-date, next-day clinic, back-to-back call, duplicate, leave, eligibility, rest, and policy checks before approval.

7. Rule and Optimization Model

Rules shall be classified as hard, overrideable hard, warning, or optimization objective. Each rule has an owner, scope, effective dates, version, severity, explanation, and test cases.

- Legal/program safety, approved leave, staffing, and eligibility precede preferences.
- Protected clinics and didactics apply from institution or resident profiles.
- Fairness compares configurable peer groups rather than unrelated roles.
- Chief overrides require authority, reason, and immutable audit history.
- Published schedule corrections create a new version and notifications.

8. Conceptual Data Model

Entity	Purpose
Program / academic year / block	Scheduling scope, dates, deadlines, lifecycle
Resident / institution	Facts, patterns, eligibility, and overrides
Rotation / master assignment	Annual placement and capacity
Service template / role / location / assignment type	Reusable monthly scheduling model
Request / protected event / decision	Resident input and controlled approval
Schedule version / daily assignment	Editable and published operational records
Call switch / eligibility result	Controlled exchange workflow
Audit event / notification	Traceability and communication

9. Nonfunctional Requirements

- **Security:** encryption, least privilege, secure sessions, dependency and vulnerability management.
- **Privacy:** minimum necessary access, restricted sensitive notes, audited exports, retention and deletion policy.
- **Reliability:** atomic approval/publication, backups, recovery, optimistic locking, and immutable history.
- **Performance:** responsive editing and visible progress for generation jobs.
- **Accessibility:** WCAG 2.2 AA target, keyboard use, contrast, screen-reader labels, and zoom.
- **Maintainability:** modular rule engine, versioned APIs, automated tests, documented migrations, and observability.

10. Implementation Architecture

- Authenticated web client for chief and resident portals.
- Application API enforcing tenant, program, role, and record-level access.
- Relational database with normalized scheduling and audit records.
- Rule-evaluation service and asynchronous optimization worker.
- Notification worker and object storage for import/export artifacts.
- Monitoring, error tracking, security audit logging, backups, and deployment pipeline.

11. Acceptance and Testing

- Configure a program and custom services without code changes.
- Import residents and institution patterns with validation preview.
- Approve requests and prove pending requests do not affect generation.
- Build a valid master schedule with capacity and transition feedback.
- Generate distinct service schedules with independent rules and rosters.
- Edit assignments and immediately recalculate coverage and fairness.
- Publish schedules with correct private/shared resident access.
- Complete an approved call switch and preserve original history.
- Export schedules and retrieve a complete audit trail.

12. Delivery Roadmap

Phase	Output
0. Discovery	Policy workshops, data dictionary, security review, signed requirements
1. Foundation	Identity, program model, profiles, services, rules, imports
2. Master schedule	Annual grid, capacity, preferences, validation, versions
3. Scheduling MVP	Generator, editor, coverage, analytics, publishing, export
4. Resident workflows	Private views, requests, leave, notifications
5. Call switches	Marketplace, eligibility, approval, controlled correction
6. Production hardening	SSO, monitoring, backups, accessibility, migration

13. Open Decisions

- Hard versus overrideable rules by service.
- Fairness peer groups and objective weights.
- Duty-hour policy source and calculation window.
- Approval authority for requests, leave, switches, publication, and correction.
- Privacy rules for special circumstances and published rosters.
- SSO, hosting, retention, backup, and recovery requirements.
- Authoritative migration source and pilot integrations.

14. Definition of Ready

- Named product owner and sponsor.
- Approved MVP and pilot program.
- Validated service templates and rule classifications.
- Approved data dictionary and import templates.
- Security/privacy requirements completed.
- Acceptance scenarios converted to a prioritized backlog.
- Synthetic test dataset available.